Document Number:	P0912R4
Date:	2019-01-16
Audience:	WG21
Revises:	P0912R3
Reply to:	gorn@microsoft.com

Merge Coroutines TS into C++20 working draft

Abstract

This paper proposes merging Working Draft of Coroutines TS [N4775] into the C++20 working draft [N4791].

Revision history

r0: initial revision

r1:

- Markdown rendered as HTML.
- Replaced ligature ff with two letters ff.
- Expanded on compiler availability.
- Reworded the closing paragraph.
- Make insertions green.
- Updated working draft numbers

r2:

- ... keywords s/were/are added ...
- P0664R3 => P0664R4
- Added list of edits requested by LWG on 2018-06-08.

r3:

- document numbers and edition instructions updated

r4:

- document numbers and edition instructions updated

Introduction

- Coroutines of this kind were available in a shipping compiler since 2014.

- We have been receiving and acting upon developer feedback for 5 years.
- The software built using coroutines has been deployed on more than **400 million** devices in the hands of delighted consumers.
- The software built using coroutines on Linux and Windows is powering the foundational services of Windows Azure cloud services.
- Coroutines have been used by thousands of software developers in various companies.
- We have two publicly available implementations of coroutines TS in MSVC since version 2015 SP2 (2016) and clang version 5 (2017).
- We have an independent coroutine implementation in EDG frontend (2015).
- GCC implementation is in progress.
- There are several open source coroutine abstraction libraries, including an extensive open source coroutine library cppcoro that utilizes most of the coroutine features with tests that can be used to smoke test emerging coroutine implementation for completeness and correctness.
- Other libraries provides coroutines TS bindings for its types, for example, Facebook's Folly, Just::Thread Pro and others.

Coroutines address the dire need by dramatically simplifying development of asynchronous code. Coroutines have been available and in use for 5 years. We have shipping implementations from two major compiler vendors. It is time to merge Coroutines TS to the working paper.

Wording

- Apply coroutine wording from N4775 to the working draft with the following changes:
    - replace `experimental::` with nothing
    - replace `<experimental/coroutine>` with `<coroutine>`
    - in synopsis in [coroutine.trivial.awaitables] and [support.coroutine]
        - remove "namespace experimental {" and "} // namespace experimental"
        - remove "inline namespace coroutines_v1 {" and "} // namespace coroutines_v1"

- LaTex specific instructions: before applying the changes, in the coroutine TS LaTeX source, replace all occurrences of \cxxref{*stable-name*} with \ref{*stable-name*}.

- Add underlined text to rationale in [diff.cpp17.lex]:

> **Rationale:** Required for new features.
> ...
> (1.5) -- ... The `co_await`, `co_yield`, and `co_return` keywords are added to enable the definition of coroutines (\ref{dcl.fct.def.coroutine}).
> **Effect on original feature:** Valid ISO C++ 2017 code using `char8_t`, `concept`, `consteval`, `co_await`, `co_yield`, `co_return`, or `requires` as an identifier is not valid in this International Standard.

- Add underlined text to "Effect on original feature" in [diff.cpp17.library]:

Effect on original feature: The following C++ headers are new: `<bit>`, `<compare>`, `<contracts>`, `<coroutine>`, `<span>`, `<syncstream>`, and `<version>`. Valid C++ 2017 code that #includes headers with these names may be invalid in this International Standard.